

Behavioral Synthesis of ICs

Computing With Uncertainty

Credentials: Arash Ahmadi

Outline

- Introduction
- Arithmetic Characteristic
- Number Representation Systems
- Data Bit-Width
- Analyzing Finite Precision Effects
- Error Bound Analysis Approach
 - Interval Arithmetic
 - Affine Arithmetic
- Noise Analysis Approach
 - Noise Model
 - Noise Propagation

Introduction

Introduction

- **Uncertainty** in computation is always a possibility with the practical problems.
- From a computing system standpoint, the origin of this uncertainty can be **external** or **internal**.
- External uncertainties are inherited from **data acquisition** or any **preceding system** which are imported together with the **input data** in the form of input **error** or **noise**.
- **Internal uncertainties**, on the other hand, arise from the current processing system because of **computation limits** and **constraints**.
- Since in many applications one or both kinds of these uncertainties are inevitable, the error effects of these phenomena on the computation process needs to be investigated.

Introduction

- Due to **data dependency**, computing the exact **ranges of the output(s)** variation in a sequence of operations is a complex task particularly when there are dependencies between intermediate variables.
- **Theoretically** the only general method to figure out the exact range is an **exhaustive simulation** of all the possible input vectors of the system.
- Since full cover of the input space is impossible in practice, usually estimates of the ranges are obtained through **partial simulation** of the input space.
- However, these estimates are not guaranteed to include all possible results and it must be assumed that the **stimuli vectors** truly represent the expected input values.
- **Accuracy analysis** of the computational systems has been investigated, with many different approaches which can be classified in many different ways, and one such classification is static analysis versus dynamic analysis.

Introduction

- **Dynamic analysis** methods are based on the use of a set of precisely chosen input stimuli to cover the **worst cases of uncertainty** in the data computation, whereas **static analysis** relies on **analytical models of the error** that functional units produce and the way that errors propagate through the system to the output.
- Since **dynamic** methods are **very time consuming** and based on **simulation**.
- Error models, on the other hand, either have an **error-bound symbolic** specification or a **statistical specification**.
- Error bond methods make it possible to keep track of the errors from their origin to eliminated or emerge those which come from the same source.
- Major symbolic error models are reviewed including **Interval Arithmetic** (IA) and **Affine Arithmetic** (AA)

Introduction

- **Statistical noise analysis** is another approach which is inspired by analogue circuit noise analysis.
- In this approach, **computational errors** are considered similar to **independent random noise** sources.
- Each functional unit has an **independent random noise generator** in its output which produces a noise with magnitude that is dependent on the maximum computational error in that functional unit.
- These noise signals **propagate** through the system exactly like signals from independent sources.
- This method is very useful in LTI DSP applications; however, it does not consider the error dependency.
- It also requires some modifications for nonlinear systems.



Arithmetic Characteristic

Arithmetic Characteristic

- **Arithmetic** units are the **building blocks** of any hardware implementation of numerical algorithms. Their **performance** and **cost** are, therefore, directly reflected in the final **implementation cost of the algorithm**.
- From a design style viewpoint, a large number of alternatives are available to realize arithmetic operations which offer different characteristics with different implementation costs. The way numbers are represented, for instance, has a great impact on the computation accuracy and the implementation cost.
- In addition to the number representation system, there are other characteristics which need to be decided for arithmetic operations in the target hardware, such as **operand word-length**, **rounding method**, **overflow prevention** and so on.
- Choosing the most suitable set of these characteristics for the arithmetic units has a vital role in the computational performance of the final hardware, they need, therefore, to be taken into account before or during the design and optimization procedure.

Arithmetic Characteristic

- To treat the issue more formally, we would like to encapsulate these characteristics as Arithmetical Characteristics, which here means a set of arithmetic or computational characteristics which attribute the arithmetical structure of a functional unit.
 - These characteristics include but not limited to:
 - Number Representation System: fixed-point, floating-point, . . . ;
 - Data Bit-Width;
 - Rounding Method: Truncation, Rounding, . . . ;
 - Overflow Prevention Method: Non-prevented, Saturated Arithmetic, . . . ;
- Clearly choosing each set of characteristics is a tradeoff of costs and benefits.

Arithmetic Characteristic

- The designer also should consider the compatibility of these characteristics in a design and also the conversion cost of exchanging data between arithmetic units which have different characteristics.
- Normally, relying on experiences, design restrictions/requirements and available Intellectual Property (IP), the characteristics of the arithmetic units are chosen at the very beginning of the hardware design procedure and these characteristics, as a generally accepted view, are applied homogenously to all functional units in the hardware; however, it is a matter for discussion whether this is the best way of doing this in reality.
- **Elementary functions** such as logarithmic or trigonometric functions are examples which are widely used in many calculations.
- Since these kinds of functions are more complicated in terms of their hardware implementation and might be used repeatedly, they need to be evaluated with more precise methods to consider different efficiency issues.



Number Representation Systems

Number Representation Systems

- **Computer arithmetic** is concerned with numbers and the basic arithmetic operations.
- The **number representation system** is considered as a vital issue in computer arithmetic.
- The choice of how real-world values are going to be represented in a digital format has a great impact on the **complexity** and **performance** of the **hardware** that implements the computation algorithms.
- Design **area**, **speed**, **power consumption**, **accuracy**, **compatibility** and data interfaces are among the aspects to take into account when a representation system is chosen.

Number Representation Systems

- In conventional digital computers, numbers are represented in the form of a finite and **ordered set of binary values** such that

$$X \leftrightarrow (x_{n-1}, x_{n-2}, \dots, x_1, x_0),$$

- where X is a n -digit number and in a radix- r representation $x_i \in [0 \ r - 1]$.
- Depending on how the original value is related to this **ordered sequence** of numbers and how can it be reconstructed from its representation, different arithmetic systems have been introduced.
- **Arithmetic systems** can be categorized into several subsets regarding the type of the numbers they represent namely: natural, integer and real numbers.

Number Representation Systems

→ Natural numbers are known as non-negative integers, which are represented in the form of weighted systems or a Residue Number Systems (RNS).

→ In a **weighted representation system**, any natural number X can be represented in the form

$$X = \sum_{k=0}^N x_k \times r^k,$$

→ where the coefficients x_i are natural numbers $0 \leq x_i < r$. It can be proved that this representation is unique for X .

→ RNS representation is defined by a set of s moduli $\{m_i\}$. If the m_i are pairwise prime for all i , the RNS is called nonredundant. The RNS representation of a given natural number X is the vector $R(X)$, whose components x_i are the respective residues modulo m_i , that is successive remainders of the integer division $\frac{X}{m_i}$ as

$$x_i = (X \bmod m_i)$$

Number Representation Systems

- Since the only difference between **natural numbers** and **integers** is their **sign**, the most natural way of representing integers is the **sign-magnitude representation system**; nonetheless, it is not the most convenient for executing arithmetic operations.
- Other common methods which should be mentioned here are **Excess-E**, **r's complement** and **Booth encoding**.
- With **sign magnitude**, an integer can be represented in the form of $+X$ or $-X$, where X is represented as a natural number and the sign can be represented using an additional sign bit. This representation is very close to a human representation of integers.
- Another way of representing negative numbers is associated with natural numbers by a defined relation such as $R(X)$, where $R(\cdot)$ is a one-to-one transform such as:

$$R(X) = \begin{cases} X & : X \geq 0 \\ X + E & : \textit{otherwise} \end{cases}$$

- where E is a predefined constant.

Number Representation Systems

- In the **Excess-E system** the range of the represented numbers is $-E \leq X \leq r^N - E$.
- r 's complement on the other hand, can be thought of as another form of $R(X)$ transform of the integer numbers, where $R(X)$ in this case is associating natural numbers to X as

$$R(X) = (X \bmod r^N),$$

- in other words, X is represented as

$$R(X) = \begin{cases} \sum_{i=0}^{N-1} x_i \times r^i & : x_{N-1} \geq \frac{r}{2} \\ \sum_{i=0}^{N-1} x_i \times r^i - r^N & : x_{N-1} < \frac{r}{2} \end{cases}$$

- here r is assumed to be even.

Number Representation Systems

- **Booth's encoding** is a signed digit method. Assuming a binary representation ($r = 2$), an integer X in this method is represented by a set of digits $X \leftrightarrow (y_{N-1}, \dots, y_1, y_0)$ which are defined based on a 2's complement representation of $X \leftrightarrow (x_{N-1}, \dots, x_1, x_0)$ as

$$\begin{aligned} X &= \sum_{i=0}^{N-1} y_i \times 2^i, \\ y_0 &= -x_0, \\ y_i &= -x_i + x_{i-1}, \quad i = 1, 2, \dots, N-1. \end{aligned}$$

- clearly $y_i \in \{-1, 0, +1\}$ holds.
- The Booth representation can be extended to a Booth- r representation, more details are available in literature¹.

1. J.-P. Deschamps, G. J. A. Bioul, and G. D. Sutter. Synthesis of Arithmetic Circuits: FPGA, ASIC and Embedded Systems. Wiley-Interscience, 2006. ISBN 0471687839.

Number Representation Systems

- Representation of real numbers, theoretically, requires an (unrealistic) infinite precision numbering system.
- To approximate real numbers there are two common numeration systems:
 - Fixed-point
 - Floating-point.
- The **fixed-point system** is an extension of the integer representation system including a decimal point representation; it provides a **relatively limited range of numbers** with a constant precision.
- Conversely, the **floating-point system** allows the representation of **a large range of numbers** with relatively higher precision.

Number Representation Systems

→ In a fixed-point numeration system, a real number X is represented as

$$X = \sum_{k=-N_1}^{N_2} x_k \times 2^k,$$

→ Let assume that X_{min} and X_{max} are the minimum and maximum numbers which can be represented by N bits respectively.

→ In a fixed-point representation any real number $\hat{X}$ belonging to the interval

$$r^{-p} \times X_{min} \leq X \leq r^{-p} \times X_{max}$$

can be represented as above with an error equal to the absolute value of the difference between $\hat{X}$ and X .

→ From another viewpoint, the distance between two succeeding numbers is one unit of the least significant position (*ULP*).

→ Accordingly, the maximum quantization error in this method is equal to $\frac{ULP}{2} = \frac{r^{-p}}{2}$.

Number Representation Systems

- **Floating-point numbers** provide a **dynamic range** of representation for real numbers without the need to scale the operands during operations.
- In a floating-point numeration system, the representation consists of two numbers: a fixed-point number (the **mantissa**) and an integer (**exponent**).
- Formally, in a **floating-point system** of radix- r , mantissa length of p (**precision**) and exponent range of $[E_{min}, E_{max}]$, a number X is represented by a mantissa

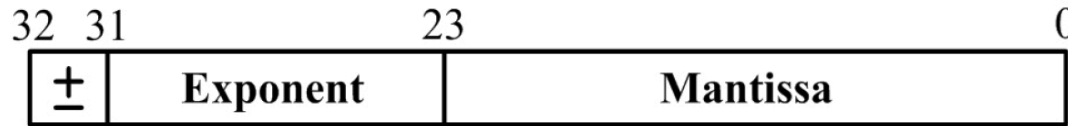
$$M_x = x_{p-1}x_{p-2} \cdots x_0$$

- which is an p -digit number in radix r , satisfying $0 \leq M_x < r$, a **sign** $S_x \in \{0, 1\}$, where the sign bit is '0' if the number is positive and '1' if the number is negative, and an exponent E_x , $E_{min} \leq E_x \leq E_{max}$ such as

$$X = (-1)^{S_x} \times M_x \times r^{E_x}$$

Number Representation Systems

- the **IEEE 754 standard** defines binary representations for **32-bit single-precision** and **64-bit double-precision** numbers as well as extended single-precision and extended double-precision numbers.
- Similar to the definition, a single-precision floating-point number presentation consists of three parts: the **sign bit**, the **exponent**, and the **mantissa**.



- The **exponent** is an 8-bit number that ranges in value from -126 to 127 . The exponent is not a typical 2's complement representation because this would make comparisons more difficult. Instead, the value is represented by an Excess-E system ($E = 127$), which makes it possible to represent negative numbers.
- The mantissa is the normalized binary representation of the number to be multiplied by 2 raised to the power defined by the exponent.

Number Representation Systems

- **Floating-point representation** can provide **more accuracy** in computation, however, due to implementation constraints such as silicon **area**, **latency** and **power consumption**, many of the floating-point algorithms need to be converted to their fixed-point version for practical reasons.
- This requires data range evaluation of the intermediate variables in the algorithm.
- Computing the exact ranges in a sequence of operations is a complex task especially when there are **dependencies between variables and operations**.
- The obvious method is an **exhaustive simulation** for all possible input values and tracing data range propagation through the computation tree of the algorithm.
- This approach is unfeasible in practice due to the size of the input space vectors; thus **analytical methods** have received more attention in the community so far.

The background is a vibrant, abstract digital scene. It features glowing binary digits (0s and 1s) in various sizes and colors (blue, yellow, red) scattered across the frame. Several bright, curved lines of light in red, orange, and blue sweep across the image, creating a sense of motion and depth. The overall color palette is dominated by dark blues and blacks, with the glowing elements providing high contrast.

Data Bit-Width

Data Bit-Width

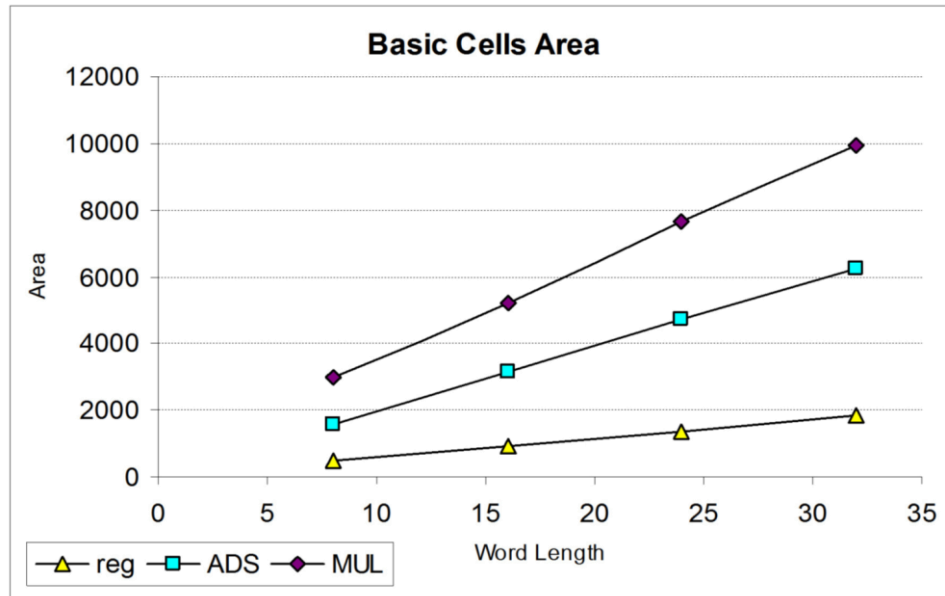
- The specification of a computation **algorithm** may not necessarily express the **ranges of the data** and intermediate results.
- However, such information is essential to determine the **bit-widths** of the **hardware resources** required for datapath synthesis.
- In practice, the computation cost including silicon **area**, **latency** and **power consumption** of a datapath depend directly on the bit-widths of the **functional units**, **communication paths** and **storage resources**.
- The **input ranges** or **bit-widths** may be part of the specification and the output is restricted to a minimum required accuracy, whereas the bit-widths requirements of the **intermediate variables** usually need to be selected by the hardware designer.

Data Bit-Width

- It is proved in that there is a tight **relationship** between the **hardware implementation cost** of the functional units and their **bit-width**.
- As it is shown in the next slide, there is an approximately linear **dependency** between implementation costs of basic functional units (register, adder, subtractor and sequential multiplier) and their bit-width.
- **Computational accuracy**, on the other hand, is directly defined by the bit-width of the operations.
- This contradiction between accuracy and implementation cost is the core idea of **bit-width (word-length) optimization**.
- The optimum **selection of the bit-width** (or word length) may imply range reduction due to implementation constraints which may result in loss of computation accuracy. However **tradeoff is tolerable in many cases**, the same accuracy might be achievable in most cases.

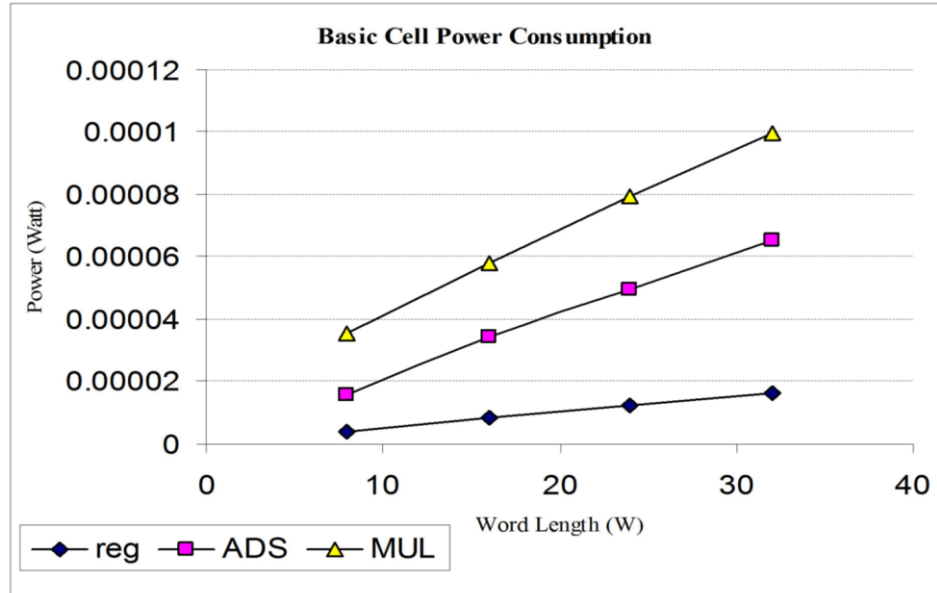
Data Bit-Width

→ **Figure 1:** Dependency of area on word length for basic cells (hardware synthesis results for Registers, Adder and Sequential Multiplier).



Data Bit-Width

→ **Figure 2:** Dependency of area on word length for basic cells (hardware synthesis results for Registers, Adder and Sequential Multiplier).



Data Bit-Width

- The **elementary functions** (e^x , $\sqrt[n]{x}$, $\ln(x)$, $\cos(x)$, $\cos^{-1}(x)$, ...) are the most commonly used mathematical functions which are differentiable and cannot be represented by finite degree polynomials without losing accuracy.
- Computing these functions with the required **accuracy** and a reasonable implementation cost is an important issue in hardware design.
- From a computer arithmetic viewpoint, this problem ranges from theoretical discussions in mathematics and algorithm design level to implementation difficulties in software and/or hardware.
- **Cost effective implementation** of the elementary functions is often the performance bottleneck of many intensive computational applications.

Data Bit-Width

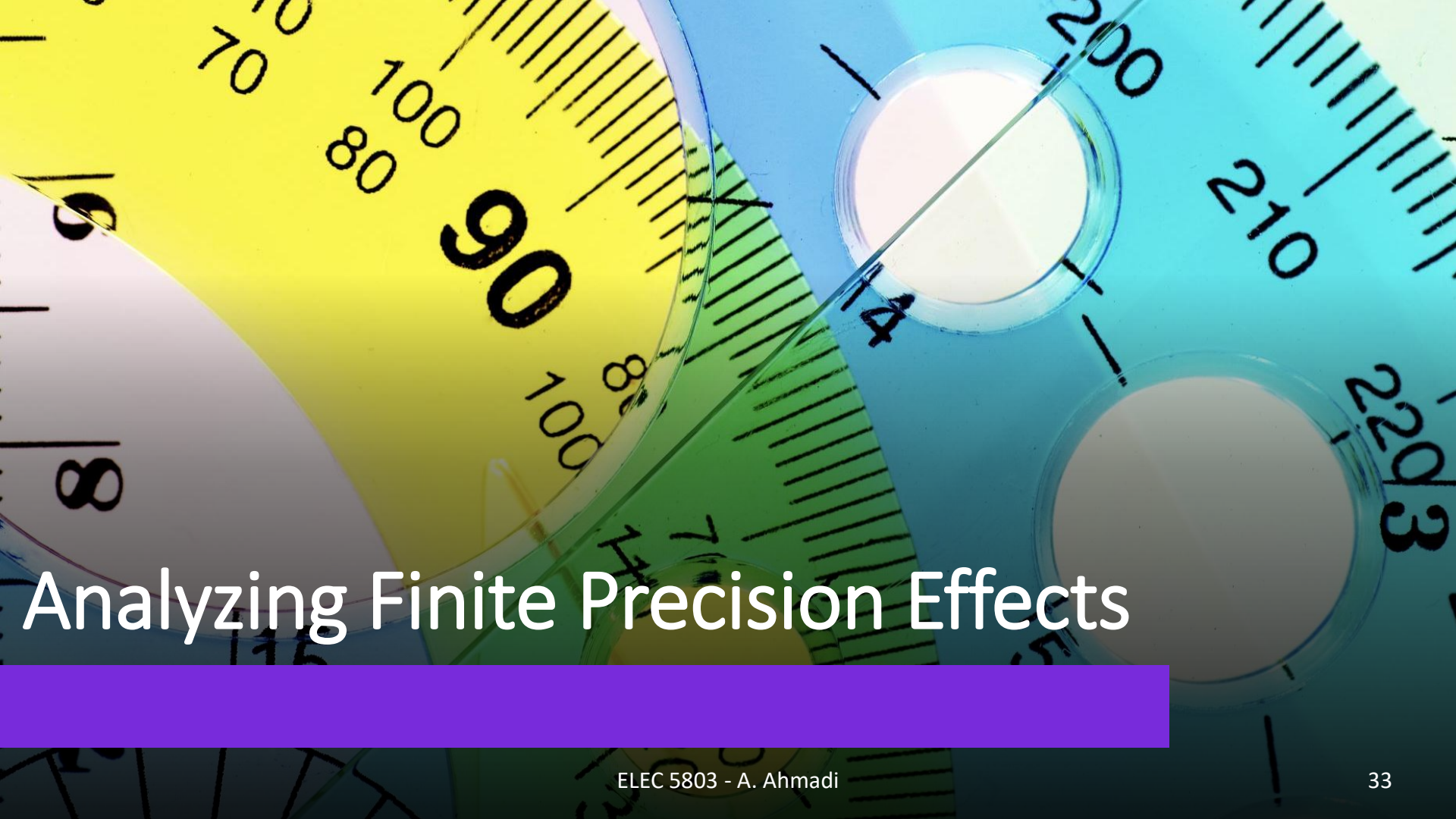
- Theoretically, the **elementary functions** are not much harder to compute than quotients in **division algorithm**.
- It is proved that elementary functions are equivalent to division with respect to **Boolean circuit depth**.
- This means that, roughly speaking, a circuits can be designed to compute N -digit elementary functions in a time proportional to $\log(N)$.
- In a more **accurate** way if it is assumed that $f(x)$ is an **elementary function** and $M(N)$ is the number of **single-precision operations** required to multiply N -bit integers; it is shown that $f(x)$ can be evaluated, with relative error $O(2^{-N})$, by $O(M(N) \log(N))$ operations as $N \rightarrow \infty$, for any floating-point number x (with an N -bit fraction) in a suitable finite interval.
- **Range reduction**, as the first step of computing an elementary function, has a great impact on evaluation accuracy.

Data Bit-Width

- A poor range reduction method may lead to **catastrophic accuracy** problems when the input arguments are large.
- In practice, two kinds of range reduction are possible: **Additive** and **Multiplicative**.
 - **Additive range reduction** means the actual value x is approximated with $\hat{x} = x - kC$, where k is an integer and C is a constant which is chosen regarding the characteristics of $f(x)$.
 - **Multiplicative range reduction** means that $\hat{x} = \frac{x}{C^k}$, where k is an integer and C is a constant which is chosen considering the characteristics of $f(x)$.
- Combinations of these basic methods with multiple C_i constant values also can be applied to improve the range reduction in special cases.
- Choosing the range reduction method depends on the **function characteristics**, for example additive range reduction is normally applied for **trigonometric functions** whereas multiplicative range reduction is more suitable for **exponential** and **logarithmic functions**

Data Bit-Width

- In modern high-level synthesis, the **computational complexity** of hardware implementation should be taken into account during minimizing the cost of the **arithmetic operations**.
- From this viewpoint, classic **computational complexity** is arguable in that computational cost of the algorithms cannot be taken down to the elementary arithmetic operations because their cost might vary depending on the implementation styles in the **target hardware**.
- Accordingly, the **arithmetic characteristic** is introduced as a design optimization parameter, especially in the case of **computationally intensive hardware**, such as hardware accelerators for DSP and AI applications.
- **Floating-point** to **fixed-point** conversion and **word-length optimization** need to be comprehensively included in the design and cost analysis as closely related fields in which the arithmetic characteristic of functional units are the foremost optimization parameter in high level synthesis.



Analyzing Finite Precision Effects

Analyzing Finite Precision Effects

- Digital hardware can only provide a limited number of binary digits for all the numbers in **computational algorithms**.
- This limitation affects the represented values in two ways: **discretization** and **bounding**.
- **Discretization** embodies an **accuracy limit** which indicates the **minimum distance between two neighbor values** in the sequence of represented numbers, whereas the **bounding limit** refers to the **maximum and minimum values** which can be represented by this hardware.
- When real values must be fitted to this constraint the result will be the original value along with an unwanted, and normally unknown, value considered as **error** or **noise**.
- In most practical computation systems, there are many of this kind of error picked up and carried by data from point to point through the **computation tree of the algorithm**.

Analyzing Finite Precision Effects

- From a systematic viewpoint, uncertainty in computation may originate from two sources: external and/or internal sources.
- Since input data of every computational system either comes from another computational or a **data acquisition system** there is always a possibility that it carry some errors due to the finite resolution of the previous system.
- **Quantization** of the constants, for example filter coefficients in DSP or synaptic weights in NN, are well-known examples of this type of error. **Internal uncertainties**, on the other hand, arise from the current processing system because of **computation limits** and **constraints**.
- **Rounding** or **truncation** of the results in arithmetic operations is an example of this kind of error. Since in many applications one or both kinds of these uncertainties are inevitable, the computation process needs to be reinvestigated concerning error effects of these phenomena.
- **Modelling** and dealing with these **uncertainties** are subjects of variety of disciplines ranging from **numerical analysis** to **computer software** and **hardware design**.

Analyzing Finite Precision Effects

- As a matter of fact, it is not trivial to analyze the **finite precision errors** in actual designs. **Computational errors** in functional units are data dependent in that different input data sets can produce different patterns of errors in the system.
- **Statistical distribution** of the input data, therefore, can cause a great impact on the actual accuracy of the system.
- Furthermore, since **finite precision effects** are nonlinear, it is observable that **computational errors in algorithms** can be dependent on the sequence of the local functions such as arithmetic operations.
- It means that during the high-level synthesis process, subsystem **allocation** and **scheduling** might affect the **predicted error** in the output if this dependency has not been considered.
- This complexity of the problem has inspired the research community to introduce different methods of error analysis and design optimization.
- There are a variety of approaches to **word-length optimization** in high level synthesis in this regard, which can be classified in many different ways.

Analyzing Finite Precision Effects

- **Dynamic analysis** relies on the use of **stimuli input data sets**. This method usually is performed by applying a set of possible inputs to the system and continuously comparing its calculated results with an auxiliary program which is performing the same algorithm but in high precision arithmetic.
- Since this approach directly compares the overall accuracy of the system with an ideal calculation, when compared to **static-analysis techniques**, it potentially provides bit-width(s) estimation for functional units close to the optimal set for those particular stimuli.
- However, in complicated systems, **number of possible inputs** (2^n vectors for an n input system) can be so high as to make it impossible to exhaustively search the input space for worst case errors.
- Hence this method can be problematic because a large set of stimuli inputs is required to analyze a complex design with sufficient confidence, possibly leading to prohibitively long simulation times and without guarantees for alternative input stimuli encountered in practice.

Analyzing Finite Precision Effects

- **Static analysis** is often more attractive than dynamic analysis especially for large designs, since only the characteristics of the input signals are needed, however, it is believed to give more conservative **bit-width estimates** than **dynamic analysis**.
- This method is based on an **analytical model of the errors** and their **propagation scheme** in the computation tree of the algorithms.
- **Errors** can be modelled in the form of **random values** with a known probability **Density Function (PDF)**, or conversely variation range of the data might be modelled as number intervals or ranges.
- Either way, the **propagation scheme** of these models must be inspected to find out the output value of the error or data range and its relationship with the local error sources.
- In spite of its simplicity, there are difficulties which this method needs to cope with.

Analyzing Finite Precision Effects

- In most of the **computational noise analysis** methods, such as machine learning and digital signal processing applications, **error sources** are considered as **independent Wide-Sense Stationary** (WSS) noise sources with uniform PDF.
- This assumption has a great impact on the method efficiency; however, there are arguments about different issues in this regard.
- For instance, in many practical computations **intermediate results** are **strongly dependent** on each other which can violate the independency assumption of the error sources.
- **Error propagation model** in **static methods** is also an important issue. Inspired by system theory studies, the primary works in this field were based on a linear Time Independency (LTI) assumption of the computational systems.
- In practice, nevertheless, many of the **computational algorithms** result in **nonlinear** system specifications. Consequently, applying methods that are designed for LTI systems cannot be used.
- Furthermore, there are applications that are required to be implemented in the form of **time dependent or adaptive systems**. In these cases, the LTI model is not valid and cannot be applied to the **error analysis method**.
- Finally, considering **HLS methods** in combination with **computational error analysis** raises a new set of problems for these analysis methods.

Error Bound Analysis Approach

Error Bound Analysis Approach

- Understanding of the required **range of variable values**, at different points of a given computation procedure, is vital to compute the **minimum bit-width requirements** in a hardware implementation of an **algorithm**.
- It is also gaining more importance in the customized hardware design methodologies, where underlying substrates allow construction of customizable **variable bit-width datapaths**.
- Assuming that the **variation ranges** of the input variables are known, this approach tries to predict the variation range of the output data.
- Therefore, instead of a single value, every number is represented by a range of values between the upper and lower bounds.
- In other words, regardless of the real place of the number in the range, these methods are concerned with **data range dilation and contraction** by **data propagation** through the operations in the **computation tree of the algorithm**.

Error Bound Analysis Approach

- These **data ranges** are normally presented in the form of **symbolic specifications**.
- Since this method is focused on **value bounds**, there is no information about how the actual value might be placed in the range.

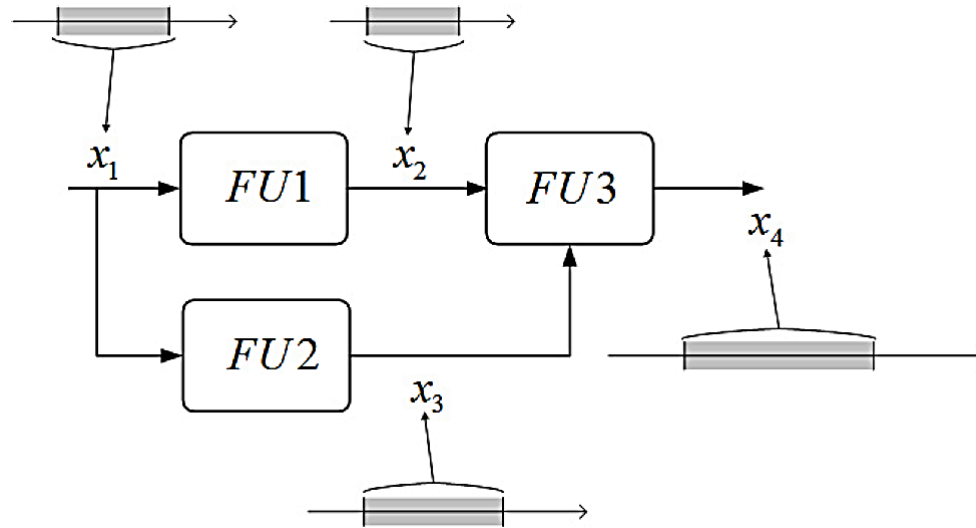


Figure 3: Variables bound propagation through the algorithm computation tree.

Interval Arithmetic

- **Interval Arithmetic (IA)** was introduced in 1960's by Ramon E. Moore as a PhD thesis at Stanford University and later on, in his works as a tool for automatic control of the errors.
- In a computed result, that arise from input error rounding errors during computation, and **truncation errors** from using a **numerical approximation** to the mathematical problem. In his study not only the foundation of interval arithmetic has been laid; it also offers interval methods for a variety of problems, such as interval version of Newton's method and automatic differentiation and algorithms for the validated solution of Ordinary Differential Equations (ODE).
- IA is a **data range analysis** method in numerical computation from which every real value x is represented by an interval $\hat{x} = [x_l, x_h]$, where $x_l \leq x \leq x_h$.
- Arithmetic operations are performed on these intervals in such a way that each result interval $\hat{y}$ is guaranteed to contain the value of the actual quantity y .

Interval Arithmetic

- Semantically, IA representation starts from a closed interval of numbers. Consider a set S_I of real interval numbers defined as

$$S_I = \{[x_l, x_h] | (x_l \leq x_h) \wedge (x_l, x_h \in \mathbf{R})\}$$

- where x_l is the lower bound of the interval and x_h is its upper bound, R is the set of real numbers and $\wedge$ represents the logical *AND*.
- Arithmetic operations on real numbers can be extended naturally to real interval numbers.
- Let $\circ$ denote a two operand, real, arithmetic operation. If $\hat{x}, \hat{y} \in S_I$ then the formal specification of $\circ$ is as

$$\hat{x} \circ \hat{y} = \{z | (z = x \circ y) \wedge (x \in \hat{x}) \wedge (y \in \hat{y})\}$$

Interval Arithmetic

- Specifically elementary arithmetic operations with two independent uncertain values represented in IA form as $\hat{x} = [x_l, x_h]$ and, $\hat{y} = [y_l, y_h]$ have

$$\hat{x} + \hat{y} = [x_l + y_l, x_h + y_h],$$

$$\hat{x} - \hat{y} = [x_l - y_h, x_h - y_l],$$

$$\hat{x} \times \hat{y} = \left[\min \{ (x_l \times y_l), (x_l \times y_h), (x_h \times y_l), (x_h \times y_h) \}, \right. \\ \left. \max \{ (x_l \times y_l), (x_l \times y_h), (x_h \times y_l), (x_h \times y_h) \} \right],$$

$$\frac{\hat{x}}{\hat{y}} = \left[\min \left\{ \frac{x_l}{y_l}, \frac{x_l}{y_h}, \frac{x_h}{y_l}, \frac{x_h}{y_h} \right\}, \min \left\{ \frac{x_l}{y_l}, \frac{x_l}{y_h}, \frac{x_h}{y_l}, \frac{x_h}{y_h} \right\} \right], \text{ assuming: } 0 \notin \hat{y},$$

Interval Arithmetic

- In general, for every function $y = f(x_1, x_2, \dots, x_n)$ where $f: \mathbf{R} \rightarrow \mathbf{R}$; a corresponding IA specification, $\hat{y}$, is defined such as: $\hat{y} = \hat{f}(\hat{x}_1, \hat{x}_2, \dots, \hat{x}_n)$.
- This operation returns the interval that contains all the values of $f(x_1, x_2, \dots, x_n)$, where variables $\{x_1, x_2, \dots, x_n\}$ are independent and range over the given intervals $\{\hat{x}_1, \hat{x}_2, \dots, \hat{x}_n\}$.
- Practically, it is observed that the computed **error bound** can be **overestimated** by IA because the **self-dependency** of the data values are not considered in IA.
- Since there is no trace of the error sources in the intervals, error sources which come from parallel branches in the computing tree might share some common components that are not considered in the interval computation.
- As a trivial example, consider $x - x = 0$ which holds for each $x \in [1, 2]$, but $\hat{x} - \hat{x}$ for $\hat{x} = [1, 2]$ results in $[-1, 1]$ in the IA method.
- There is also another source of overestimation which is called **wrapping effect**. This effect appears when intermediate results of a computation are enclosed into intervals.

Interval Arithmetic

→ **Example:** Consider the function $f: (x, y) \rightarrow \frac{\sqrt{2}}{2}(x + y, y - x)$.

- The image of the square $[0, \sqrt{2}]^2$ is the rotated square with corners $(0,0)$, $(1,-1)$, $(2,0)$, $(1,1)$.
- On the other hand, interval computation yields $f([0, \sqrt{2}], [0, \sqrt{2}]) = ([0, 2], [-1, 1])$, these regions are depicted in the figure below.

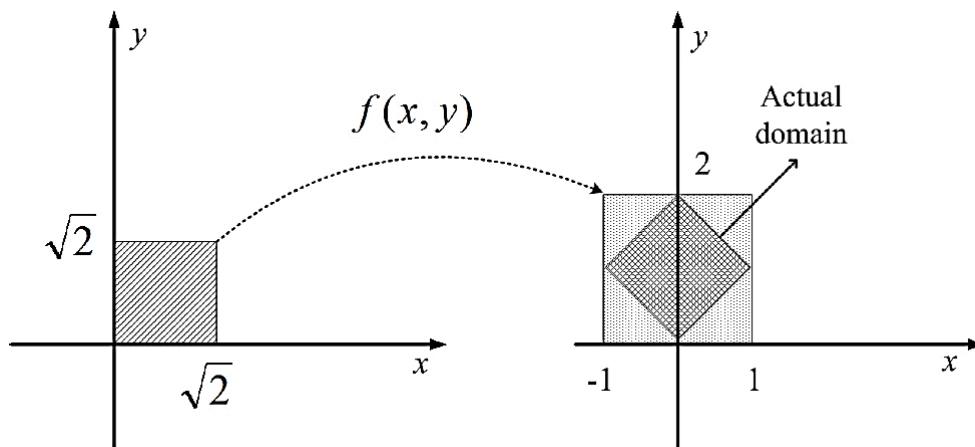


Figure 4: Wrapping problem in IA method.

Interval Arithmetic

- Note that the observed overestimation in Figure 4 (the area of the interval enclosure is twice the area of the range) is not a result of dependency, but of the enclosure of a non-interval shaped range into an interval.
- Overestimations of this kind are one of the major problems in the interval arithmetic treatment of differential equations.

Interval Arithmetic

- **IA computing**, as a pioneering method in computing with uncertainties, is a very useful method with low computation overhead.
- It can easily be implemented by overloading elementary operations with IA operations and representing numbers in the form of **ordered pairs**.
- The **major downside** of the IA is its **very conservative error range estimation**. This weak-point especially can be exaggerated in practical computation tasks, where in **long computation chains** the computed intervals in one stage are inputs for the next stages.
- This conservatism is mainly due to this assumption that the unknown values of the arguments to the **primitive operators vary independently** over the given intervals.
- If this assumption is not true, which it is not mostly and then having correlation between possible results, not all the combinations of values in the given interval will appear in the results set.
- In that case, the interval obtained by IA can be much wider than the exact range of the result quantity, which known as “**error explosion**”.

Affine Arithmetic

- **Affine arithmetic** (AA) is a method for numerical computation that aims to attack the **dependency problem in interval computations**.
- AA keeps track of **first-order correlations** between computed and input quantities; these correlations are automatically exploited in primitive operations, with the result that in many cases AA is able to produce interval estimates that are much better than the ones obtained with standard interval arithmetic.
- Moreover, AA also implicitly provides a **symbolic representation** for the joint range of related quantities that can be exploited to increase the efficiency of interval methods.

Affine Arithmetic

- A partially known quantity x in AA is represented by an affine form $\hat{x}$, which is a first-degree polynomial representation as

$$\hat{x} = x_0 + x_1\varepsilon_1 + x_2\varepsilon_2 + \cdots x_n\varepsilon_n$$

- where the coefficients x_i are finite real numbers and ε_i are symbolic real variables, whose values are unknown but assumed to lie in the interval $U = [-1, +1]$.
- x_0 is called the **central value** of the affine form $\hat{x}$.
- The coefficients x_i are **partial deviations** and ε_i are symbolic representation of the **uncertainty in variables**.
- In $\hat{x}$ equation each ε_i stand for an independent component of the total uncertainty of the quantity x . The corresponding coefficients x_i represents the magnitude of each component.
- In this definition, the uncertainty source might be either internal (arithmetic rounding for instance) or external (indeterminacy of the input data such as quantization error).

Affine Arithmetic

- To model the elementary operations this method divides all operations into two categories: Affine operations and Non-affine operations.
- The first one refers to those which result in a **linear representation** of the **error symbols** in the output whereas the later represents those which result in **nonlinear combination** of the symbols, multiplication for instance.
- If the operation $f(x, y)$ itself is an affine function of its arguments x and y , then f can be expanded and rearranged into an affine combination of the symbols ε_i .
- Specifically, for any given constants $\alpha, \beta, \zeta \in \mathbf{R}$, the computation $z \rightarrow \alpha x + \beta y + \zeta$ can be carried out by affine expansion of x and y as

$$\hat{x} = x_0 + \sum_{i=0}^n x_i \varepsilon_i, \text{ and } \hat{y} = y_0 + \sum_{i=0}^n y_i \varepsilon_i, \text{ then}$$
$$\hat{z} = \alpha \hat{x} + \beta \hat{y} + \zeta = (\alpha x_0 + \beta y_0 + \zeta) + \sum_{i=0}^n (\alpha x_i + \beta y_i) \varepsilon_i$$

Affine Arithmetic

- Except for roundoff errors and overflows, the affine form $\hat{z}$ in previous equation, together with $\hat{x}$ and $\hat{y}$, captures all the information about the quantities x , y , and z that can be deduced from the given affine forms $\hat{x}$ and $\hat{y}$, and the equation $z = \alpha x + \beta y + \zeta$.
- An AA computation that uses only affine operations with known constant coefficients will usually give an almost-exact range for the ideal result.
- In fact, it will give an **almost-exact explicit formula** for the ideal result in terms of the input variables. The most important primitive operation is not affine is multiplication.
- Given two affine numbers $\hat{x}$ and $\hat{y}$, their product is

$$\begin{aligned}\hat{x} \cdot \hat{y} &= \left(x_0 + \sum_{i=1}^n x_i \epsilon_i \right) \cdot \left(x_0 + \sum_{i=1}^n y_i \epsilon_i \right), \\ &= x_0 y_0 + \sum_{i=1}^n (x_0 y_i + y_0 x_i) \epsilon_i + \sum_{i=1}^n x_i \epsilon_i \cdot \sum_{i=1}^n y_i \epsilon_i,\end{aligned}$$

Affine Arithmetic

- The affine numbers multiplication can be rewritten as the following affine form for the product:

$$\hat{x} \cdot \hat{y} = x_0 y_0 + \sum_{i=1}^n (x_0 y_i + y_0 x_i) \epsilon_i + z_k \epsilon_k,$$

- where

$$z_k \geq \left| \sum_{i=1}^n x_i \epsilon_i \cdot \sum_{i=1}^n y_i \epsilon_i \right|, \quad \epsilon_i \in U,$$

- is an upper bound for the approximation error, as before. The simplest bound is:

$$z_k = \sum_{i=1}^n |x_i| \cdot \sum_{i=1}^n |y_i|,$$

- which is at most four times the error of the best affine approximation but is very easily computed.

Noise Analysis Approach

- **Error range propagation** methods provide an expression of the error range at the output to evaluate the **computation accuracy**.
- This approach does not provide any **statistical information** about the **error distribution over this range**.
- **Noise analysis approach**, on the other hand, gives a statistical description of the error at the output in the form of noise energy or **signal-to-noise** parameter.
- This method is widely practiced in **digital signal processing design and optimization**.

Noise Analysis Approach

- Applying noise analysis consists of two parts:
 - noise model,
 - noise propagation.
- The first one refers to the **error modelling** of the functional units and the arithmetic characteristics under finite precision effects.
- Functional unit operation type (+, -, ×, · · ·), arithmetic system (fixed-point/floating point), word-length, rounding method and overflow prevention method are the major parameters which need to be taken into account in noise modelling.
- In ideal case, every computational error in each functional unit is an **independent random noise generator**.
- To compute the overall effect of the errors at the output, therefore, these noise sources are considered to **propagate through the system like signals** and their spectrum in the output is calculated using noise sources and the system structure.

Modelling Digital Noise

- From a mathematical point of view, computational error can be modeled in the form of an **additive error**, which means that adding or removing unwanted and indeterministic information to the input data, appears at the output as an added noise value (signal) to the original value (signal).
- In digital implementation of the (DSP) algorithms, depending on the way of interaction with these computational noises, they have been divided into two major categories: **quantization noise** and **round off error**.
- However both of these errors arise from the **finite precision limitation**, the way they affect the system are different and thus require different solutions.
- **Quantization noise** is considered as an **external source of the noise** in that input data carries it before entering the computation system, whereas **round off noise** has **internal sources** which result from the finite precision restrictions of the arithmetic units which are trying to refit the data to their finite word length.
- I prefer to call this kind of error as a “**data refit error**”.

Noise Analysis Approach

- One of the input noise sources inherited from all previous computational operations or **data acquisition systems**.
- Every error imported to the system by input data will be carried processed along side with the original information.
- Depending on the type and characteristics, these errors might be strengthened or weakened compared with the original information through data processing.
- The most known and well studied type of error which appears almost everywhere in computational systems is **quantization error**.
- This error is investigated widely in the field of DSP and coding.

Noise Analysis Approach

- Another **external source of the error** comes from **coefficient quantization**.
- Every system has a set of **constant values** representing the basic system characteristics, weights in ML or filter coefficients in DSP for instance.
- These values, however, are real numbers ($x \in R$) in reality but they must be used by a digital computing system.
- With a limited number of binary digits, **truncation** or **rounding** of these values is needed which causes an error that is considered also as quantization noise.
- From a **system analysis** point of view, when parameters of the system function (or its corresponding difference equation specification) are quantized, the **poles** and **zeros** of the system function might move to new positions in the Z-plane, which in the case of a system implementation **highly sensitive to the perturbation of the coefficients**, may mean the resulting system might no longer meet the design specifications, or in the case of IIR system even might become unstable.

Noise Analysis Approach

- In **additive noise model**, a finite word-length unit can be modelled in terms of an ideal unit followed by an additive noise source as shown in Figure 5.
- Three distinct approximation schemes can be employed after fitting the output data into the available word-length of the functional unit by **rounding**, **truncation** or **magnitude truncation**.
- Each of these approximation methods gives a different noise source **probability density function**., as shown in Figure 6.

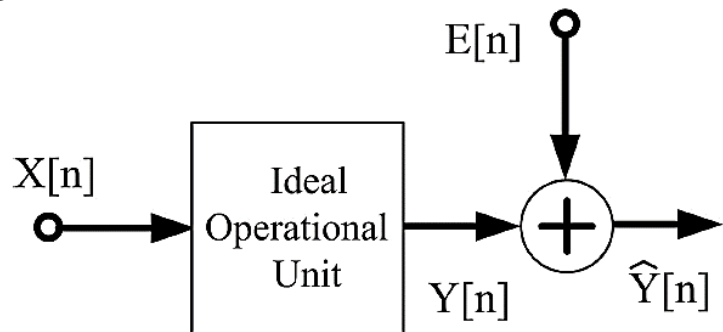
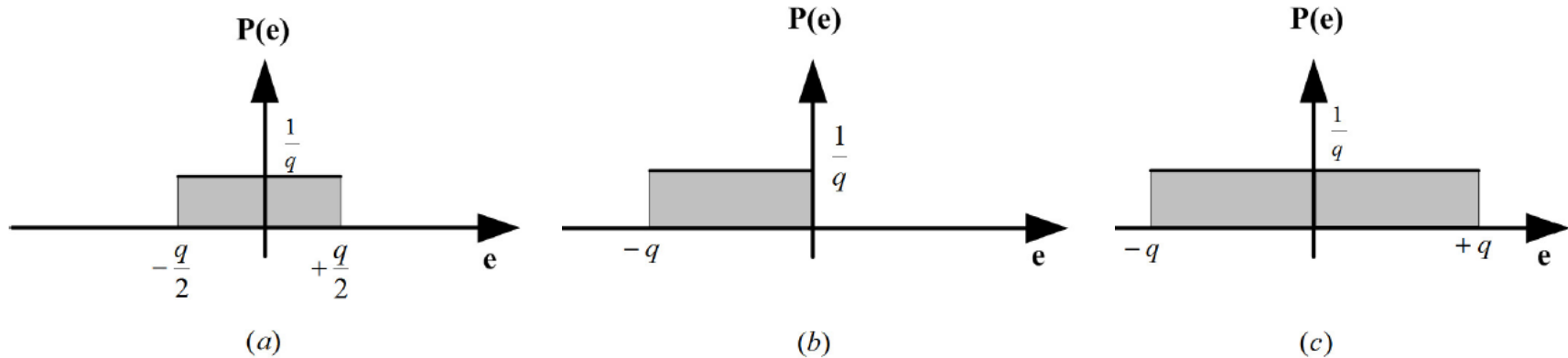


Figure 5: Additive computational noise model.

Noise Analysis Approach

→ **Figure 6:** Probability density function for quantization error a) Rounding b) 2's complement truncation c) Magnitude truncation.



Noise Analysis Approach

- In the case of **rounding**, the output number must be fitted to a finite precision number whose value is the closest possible to actual number as depicted in Figure 6-a.
- The mean or **expected value** of the quantization error due to rounding is zero and the **variance** of the error signal ($E(n)$) as

$$\sigma_E^2 = \frac{q^2}{12} = \frac{2^{-2N}}{12},$$

- where q is the **maximum value of the rounding error** and N is the word-length of the target unit such that $q = 2^{-N}$.
- In the case of **truncating** a number, where the result is always less than the original value, as depicted in Figure 6-b, the **mean value** of the error is shown in

$$E \{E(n)\} = -\frac{2^{-N}}{2},$$

Noise Analysis Approach

→ and the error **variance** is $\sigma_E^2 = \frac{2^{-2N}}{12},$

→ If **magnitude truncation** is applied to 2's complement numbers, the **probability density function** has the form of the Figure (3.5-c). In this case the mean value of the error is zero and the variance is

$$\sigma_E^2 = \frac{2^{-2N}}{3}$$

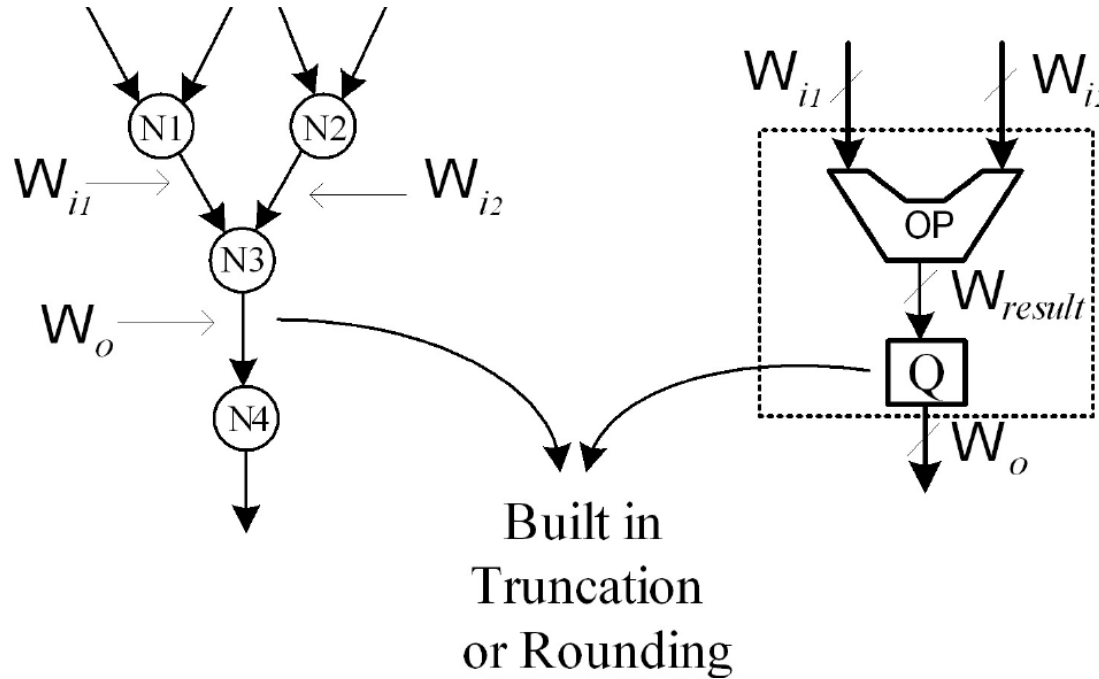
- In addition to input noise, during computation with digital systems in most of the cases after every arithmetic operation the actual result needs to be fitted into the output word length.
- This operation might be a source of error if the result requires more precision than can be represented by the output.

Noise Analysis Approach

- The round off noise is generated by the rounding or truncation operation that follows the various arithmetic operations.
- It mostly happens in the case of multiplication where the result of the product of two N -bit fixed-point fractions is a $(2N - 1)$ bit number and it must eventually be quantized to M -bits ($M < 2N - 1$) by rounding or truncation.
- In the systems with fixed word-length the truncation is done by built in units as shown in Figure 7.
- As a common assumption, these noise sources have been formulated as **independent white noise** in DSP design.
- By this assumption, knowing the **variance** and **mean value** will be enough to provide an acceptable approximation for the noise produced in the system output(s), however, this assumption necessitates some preliminary conditions.

Noise Analysis Approach

→ **Figure 7:** Built in operation output truncation off



Noise Propagation

- In addition to the **noise model**, the **propagation scheme of the noise** is required to calculate the noise effect at the output.
- From a **system analysis viewpoint**, this issue can be studied for two kinds of systems: linear and nonlinear.
- **Linear systems**, however, rarely exist in practice, but are the simplest model by which the input/output relationship is expressed with a linear relationship.
- It is also considered as the basis of many **nonlinear systems** analysis methods.
- Nonlinear models, on the other hand, in most of the cases provide more accurate but very complex models.

Noise Propagation

→ **Figure 8:**
Two Pole
IIR Filter
structure
with its
Noise
Sources.

